

CONTENTS

Emparejamiento Estable (Algoritmo de Gale-Shapley)	
Guía de estudio con tolerancia cero — Módulo 1, Programación 3 (FING UdelaR) — Kleinberg & Tardos, Cap. 1	
0. Por qué existe este problema/tema	
1. Definición formal del problema	
2. El algoritmo de Gale-Shapley	
3. Terminación del algoritmo	
4. Demostración de que el algoritmo produce un matching estable	
4.a) El conjunto devuelto es un emparejamiento (matching)	
4.b) El matching devuelto es perfecto	
4.c) El matching devuelto es estable	
5. Optimalidad para proponentes (M-óptimo) y pesimalidad para receptores (W-pesimal)	
6. Complejidad temporal: por qué es exactamente $O(n^2)$	
7. Variantes típicas de examen	
7.a) Pares prohibidos / listas incompletas (aceptabilidad)	
7.b) Cupos múltiples (Hospitales-Residentes, KT Ex. 1.4)	
7.c) Orden de propuesta o función de comparación no estándar	
7.d) Reducciones a Emparejamiento Estable	
Tabla de referencia rápida — objetos del problema	
Ejercicios de repaso rápido (con respuesta)	
Resumen ultra-rápido (2 minutos)	

Emparejamiento Estable (Algoritmo de Gale-Shapley)

GUÍA DE ESTUDIO CON TOLERANCIA CERO — MÓDULO 1, PROGRAMACIÓN 3 (FING UDELAR) — KLEINBERG & TARDOS, CAP. 1

0. Por qué existe este problema/tema

El curso arranca con Emparejamiento Estable por una razón muy concreta: es el ejemplo más simple posible donde conviven, en un solo problema, **existencia** (¿siempre hay solución?), **algoritmo constructivo** (¿cómo la encuentro?), **corrección** (¿termina? ¿lo que devuelve es lo que pido?) y **análisis de complejidad** (¿qué tan rápido?). Todo el resto del curso — programación dinámica, greedy, flujo máximo, NP-completitud — reutiliza este mismo esqueleto de preguntas.

El problema en sí es intuitivo: tengo dos grupos del mismo tamaño (hospitales/residentes, empresas/aplicantes, hombres/mujeres en la formulación clásica del libro), cada integrante de un grupo ordena totalmente a los del otro grupo por preferencia, y quiero armar parejas de forma que **nadie tenga incentivo a “hacer trampa” por fuera del sistema**: que no exista un par (m, w) no emparejado entre sí que, si se conocieran, ambos preferirían dejar a su pareja actual por el otro. A esa condición se le llama *estabilidad*, y es la que le da nombre al tema.

Lo que hace que este módulo se examine con tanta frecuencia (y con tan poca piedad) es que el **algoritmo de Gale-Shapley** es corto y fácil de memorizar, pero las **cuatro demostraciones** que lo acompañan (termina, produce un matching, es perfecto, es estable) tienen una estructura de argumento por absurdo muy particular que hay que saber reproducir con precisión — y en el examen casi nunca se pide “correr el algoritmo tal cual”: se pide adaptarlo a una variante (cupos múltiples, listas incompletas, pares prohibidos, orden de propuesta distinto) y **rehacer las demostraciones adaptadas**. Dominar la plantilla de demostración importa más que memorizar el pseudocódigo.

Regla de oro que atraviesa todo el módulo: en **toda** ejecución del algoritmo “M propone”, una vez que una mujer queda comprometida **nunca vuelve a estar libre** — solo puede *mejorar* de pareja (en términos de su propia lista de preferencia), nunca empeorar. Simétricamente, un hombre libre **solo empeora** con el tiempo: cada vez que le proponen y es rechazado, su próxima propuesta es a alguien peor en su propia lista. Estas dos observaciones son el corazón de las cuatro demostraciones del módulo.

1. Definición formal del problema

Instancia. Dos conjuntos disjuntos del mismo tamaño: $M = \{m_1, \dots, m_n\}$ (hombres) y $W = \{w_1, \dots, w_n\}$ (mujeres) — en la notación del libro también aparecen como “hospitales” (H) y “estudiantes” (S). Cada $m \in M$ tiene una **lista de preferencia total** (sin empates) sobre todo W , y cada $w \in W$ tiene una lista de prefer-

encia total sobre todo M .

Emparejamiento (matching) S . Subconjunto de $M \times W$ donde nadie aparece repetido: cada persona aparece en a lo sumo un par de S .

Emparejamiento perfecto. Un matching S es perfecto si $|S| = n$, es decir, nadie queda sin pareja (cada persona aparece en exactamente un par).

Inestabilidad (par bloqueante / rogue couple). Dado un matching S , el par $(m, w) \notin S$ es una **inestabilidad respecto de S** si se cumplen *ambas* condiciones:

- m prefiere w a su pareja actual en S (o m está libre en S), **y**
- w prefiere m a su pareja actual en S (o w está libre en S).

Emparejamiento estable. Un matching S es estable si es **perfecto** y **no tiene ninguna inestabilidad**.

La definición de inestabilidad exige **ambas** condiciones a la vez — es un AND, no un OR. Un par (m, w) donde solo m prefiere a w más que a su pareja actual (pero w está feliz con la suya) **NO** es una inestabilidad: w no tiene ningún incentivo para irse con m , así que ese par nunca se forma “en la práctica”. Confundir esto con un OR es el error conceptual más común al arrancar el tema.

Ejemplo 1.1 — Identificar una inestabilidad (instancia de las diapositivas del curso).

Hospitales $H = \{\text{Atlanta}, \text{Boston}, \text{Chicago}\}$, estudiantes $S = \{\text{Xavier}, \text{Yolanda}, \text{Zeus}\}$.

Preferencias de hospitales: Atlanta: $X > Y > Z$. Boston: $Y > X > Z$. Chicago: $X > Y > Z$.

Preferencias de estudiantes: Xavier: $B > A > C$. Yolanda: $A > B > C$. Zeus: $A > B > C$.

Matching $M = \{A-Z, B-Y, C-X\}$ (perfecto, cada uno con pareja).

Miro el par (Atlanta, Yolanda), que no pertenece a M : Atlanta prefiere a Yolanda (2do lugar) sobre Zeus (su pareja actual, 3er lugar) ✓. Yolanda prefiere a Atlanta (1er lugar) sobre Boston, su pareja actual (2do lugar) ✓.

Verificación: ambas condiciones se cumplen simultáneamente $\rightarrow$ (Atlanta, Yolanda) es una inestabilidad respecto de M . Por lo tanto M **NO** es estable, aunque sí es perfecto.

Ejemplo 1.2 — Un matching estable para la misma instancia.

$M' = \{A-X, B-Y, C-Z\}$. Reviso los 6 pares que no están en M' (A-Y, A-Z, B-X, B-Z, C-X, C-Y) uno por uno: en cada caso, o el hospital ya tiene a su primera opción, o el estudiante prefiere a su pareja actual sobre el hospital candidato. Ninguno cumple las dos condiciones a la vez.

Verificación: M' es perfecto y no tiene inestabilidades $\rightarrow M'$ es estable.

2. El algoritmo de Gale-Shapley

Versión “**M propone**” (la estándar del curso):

```

Inicializar a todos en M y W como libres
Mientras exista un hombre m libre que no le haya propuesto a toda mujer:
    w = primera mujer en la lista de preferencia de m a la que no le propuso todavía
    Si w está libre:
        (m, w) se comprometen
    Si no (w está comprometida con m'):
        Si w prefiere m a m':
            (m, w) se comprometen; m' vuelve a estar libre
        Si no:
            w rechaza a m (m sigue libre)
Devolver el conjunto S de pares comprometidos

```

Invariantes clave (se usan en TODAS las demostraciones posteriores):

1. **Los hombres proponen en orden estrictamente decreciente de su propia preferencia.** Un hombre nunca le vuelve a proponer a la misma mujer, y cada vez que vuelve a estar libre, su siguiente propuesta es a alguien que le gusta *menos* que la anterior.
2. **Una mujer, una vez comprometida, permanece comprometida para siempre** — solo puede cambiar de pareja (y siempre a alguien que prefiere más), nunca vuelve a estar libre.

Estas dos invariantes son **asimétricas por diseño**: los hombres van “degradando” su aspiración con el tiempo (proponen a peores mujeres), mientras que las mujeres solo “mejoran” (nunca aceptan a alguien peor de lo que ya tienen comprometido). Esta asimetría — consecuencia directa de que M propone y W responde — es la que después produce el resultado de M-optimalidad / W-pesimalidad de la Sección 5. Si el enunciado de un ejercicio de examen invierte quién propone, hay que invertir también esta asimetría en las demostraciones.

3. Terminación del algoritmo

Afirmación. El algoritmo de Gale-Shapley termina después de, a lo sumo, **n^2 iteraciones** del ciclo `while`.

Demostración. Cada iteración del ciclo consiste en que un hombre m le propone a una mujer w a la que **nunca antes le había propuesto** (por la invariante 1). Definamos $P(t)$ como el conjunto de pares (m,w) tales que m ya le propuso a w al terminar la iteración t. Como en cada iteración se agrega exactamente un par nuevo a P, se tiene que $|P(t+1)| = |P(t)| + 1$ para todo t. Como hay a lo sumo n^2 pares posibles (m,w) distintos (n hombres $\times$ n mujeres), el tamaño de $P(\cdot)$ puede crecer a lo sumo n^2 veces a lo largo de toda la ejecución. Por lo tanto, el ciclo `while` se ejecuta a lo sumo n^2 veces. ■

La cota n^2 es la que hay que citar en el examen porque es la que sale directo del argumento de conteo de pares (m,w). Existe una cota más ajustada y exacta — **$n(n-1)+1$** — construida con una instancia específica que fuerza ese máximo (ver Ejemplo 3.1), pero para las demostraciones del curso alcanza y sobra con justificar $O(n^2)/n^2$ propuestas como cota superior. No hace falta memorizar $n(n-1)+1$; sí hay que saber que la cota “cae” directamente de contar pares (m,w) posibles.

Ejemplo 3.1 — Verificación numérica de la cota, con simulación completa.

Instancia con $n=4$: hombres $\{m1, m2, m3, m4\}$, mujeres $\{w1, w2, w3, w4\}$, con las preferencias:

$m1: w1 > w2 > w3 > w4$ $m2: w1 > w3 > w2 > w4$ $m3: w2 > w1 > w4 > w3$ $m4: w2 > w4 > w1 > w3$
 $w1: m4 > m3 > m2 > m1$ $w2: m3 > m4 > m1 > m2$ $w3: m1 > m2 > m3 > m4$ $w4: m2 > m1 > m4 > m3$

Traza completa (verificada con un script Python que implementa el algoritmo literal y con enumeración por fuerza bruta de los $4!=24$ matchings perfectos para contrastar):

1. $m1$ propone a $w1 \rightarrow w1$ libre $\rightarrow$ se comprometen $(m1, w1)$
2. $m2$ propone a $w1 \rightarrow w1$ prefiere a $m2$ sobre $m1 \rightarrow m1$ vuelve a estar libre; $(m2, w1)$
3. $m3$ propone a $w2 \rightarrow w2$ libre $\rightarrow$ se comprometen $(m3, w2)$
4. $m4$ propone a $w2 \rightarrow w2$ prefiere a $m3$ sobre $m4 \rightarrow w2$ rechaza a $m4$
5. $m1$ propone a $w2$ (su 2da opción) $\rightarrow w2$ prefiere a $m3$ sobre $m1 \rightarrow w2$ rechaza a $m1$
6. $m4$ propone a $w4$ (su 2da opción) $\rightarrow w4$ libre $\rightarrow$ se comprometen $(m4, w4)$
7. $m1$ propone a $w3$ (su 3ra opción) $\rightarrow w3$ libre $\rightarrow$ se comprometen $(m1, w3)$

Total de propuestas: 7. Matching final: $\{m1-w3, m2-w1, m3-w2, m4-w4\}$.

Verificación de la cota: $7 \leq n^2 = 4^2 = 16 \checkmark$ (la cota se cumple con margen; no todas las instancias la saturan). Se verificó por fuerza bruta que este matching es efectivamente uno de los 2 matchings estables que tiene esta instancia (ver Sección 5, donde se reutiliza esta misma instancia).

Error frecuente de tolerancia cero: escribir “el algoritmo hace exactamente n^2 propuestas” en vez de “a lo sumo n^2 propuestas”. La cota es una cota **superior**, no un valor exacto — como muestra el Ejemplo 3.1, puede terminar con muchas menos (7 en vez de 16). En una demostración de examen, usar “ $\leq$ ” y no “=” es tolerancia cero.

4. Demostración de que el algoritmo produce un matching estable

Se demuestra en tres pasos independientes, cada uno con su propio argumento.

4.A) EL CONJUNTO DEVUELTO ES UN EMPAREJAMIENTO (MATCHING)

Por construcción: cada vez que se forma un par (m, w) , cualquier compromiso previo de m o de w se deshace explícitamente antes de formar el nuevo (m vuelve a estar libre). Por inducción en el número de iteraciones, el conjunto S de pares comprometidos es, en todo momento de la ejecución, un matching válido (nadie repetido).

4.B) EL MATCHING DEVUELTO ES PERFECTO

Lema previo. Si en algún momento de la ejecución un hombre m está libre, entonces existe alguna mujer a la que m todavía no le propuso.

Demostración por absurdo. Supongamos que m está libre y ya le propuso a las n mujeres. Por la invariante 2, toda mujer que alguna vez recibió una propuesta permanece comprometida para siempre. Como m le propuso a las n mujeres, las n mujeres están comprometidas en este momento. Pero un matching donde las n mujeres están comprometidas requiere n hombres comprometidos (es un conjunto de pares 1 a 1) — y hay exactamente n hombres en total, así que **todos** los hombres estarían comprometidos, contradiciendo que m está libre. ■

Afirmación. El matching S devuelto al finalizar es perfecto.

Demostración por absurdo. Supongamos que el algoritmo termina con algún hombre m libre. Por la condición de salida del `while`, eso solo ocurre si m ya le propuso a todas las mujeres — pero el Lema previo dice que eso es imposible mientras m siga libre. Contradicción. Por lo tanto, al terminar, ningún hombre está libre, y como $|M| = |W| = n$, tampoco ninguna mujer está libre: S es perfecto. ■

4.C) EL MATCHING DEVUELTO ES ESTABLE

Afirmación. El matching S^* devuelto por Gale-Shapley no tiene inestabilidades.

Demostración por absurdo. Supongamos que S^* tiene una inestabilidad, es decir, existe un par $(m, w) \notin S^*$ tal que:

- m prefiere w a su pareja $s' := S^*(m)$, y
- w prefiere m a su pareja $h' := S^*(w)$.

Como m prefiere w a su propia pareja final s' , y los hombres proponen en orden decreciente de preferencia (invariante 1), en algún momento de la ejecución **m le propuso a w** (antes de llegar a s' , que está más abajo en su lista). Hay dos casos sobre qué pasó con esa propuesta:

- **w rechazó a m en el momento de la propuesta** porque en ese momento ya estaba comprometida con alguien que prefería más que m . Por la invariante 2, la pareja de w solo mejora con el tiempo, así que la pareja final h' de w es al menos tan buena para w como aquel rechazo — en particular, **w prefiere h' a m** . Esto contradice la hipótesis de que w prefiere m a h' .
- **w aceptó a m en ese momento pero después lo dejó por alguien mejor** (y esa cadena de mejoras terminó en h'). Por la misma invariante 2, w terminó con alguien que prefiere *al menos* tanto como m , en particular **w prefiere h' a m** (o le es indiferente, pero como las preferencias son estrictas, prefiere a h' estrictamente si $h' \neq m$). Misma contradicción.

En ambos casos se llega a una contradicción con la hipótesis de que (m, w) es una inestabilidad. Por lo tanto S^* no tiene inestabilidades, y como ya se probó que es perfecto (4.b), **S es estable***. ■

Estructura para memorizar de esta demostración (aparece calcada en casi todas las variantes de examen): **(1)** asumir por absurdo que existe una inestabilidad (m, w) ; **(2)** usar que m prefiere w a su pareja final para concluir que m *en algún momento* le propuso a w (invariante 1); **(3)** usar que si w no terminó con m fue porque lo rechazó o lo dejó, y en cualquiera de los dos casos, por la invariante 2, w terminó con alguien que le gusta al menos tanto como m ; **(4)** eso contradice que w prefiere m a su pareja final. Cuando cambia la variante (cupos, listas incompletas, pares prohibidos), esta plantilla se

adapta reemplazando “pareja final” por “alguna de sus plazas ocupadas” o agregando el chequeo de aceptabilidad, pero el esqueleto argumental es el mismo.

5. Optimalidad para proponentes (M-óptimo) y pesimalidad para receptores (W-pesimal)

Definición — pareja válida. w es una **pareja válida** de m (y viceversa) si existe *algún* emparejamiento estable de la instancia en el que m y w están juntos. (Puede haber varias parejas válidas para una misma persona si la instancia admite más de un matching estable.)

Definición — asignación M-óptima. Es la asignación donde cada hombre queda emparejado con su **mejor** pareja válida (la más alta en su propia lista de preferencia, entre todas las que son válidas para él).

Teorema (M-optimalidad). El algoritmo de Gale-Shapley “M propone” siempre produce la asignación M-óptima: cada hombre termina con la mejor mujer que podría tener en *cualquier* emparejamiento estable de la instancia.

Demostración (por absurdo, versión condensada). Supongamos que no es cierto: entonces, en algún momento de la ejecución, una mujer w rechaza (o desplaza) a un hombre m que es una pareja válida para ella, en favor de otro m' . Sea E el primer momento en que esto ocurre en toda la ejecución. Como (m, w) es válida, existe algún emparejamiento estable S' donde están juntos. Sea w' la pareja de m' en S' . En S' tenemos los pares (m, w) y (m', w') . Como E es el primer rechazo de una pareja válida en toda la ejecución, hasta ese momento m' no había sido rechazado por ninguna pareja válida suya — en particular no por w' — así que en el momento E , m' todavía prefiere a w sobre w' (si no, ya habría sido rechazado por w' , su pareja en S' , contradiciendo que E es el primer rechazo de un válido). Pero además w prefiere a m' sobre m (es lo que dice que w rechazó a m por m' en E). Entonces en S' , el par (m', w) es una inestabilidad: m' prefiere w a su pareja w' , y w prefiere m' a su pareja m . Esto contradice que S' es estable. ■

Teorema (W-pesimalidad). En la misma ejecución de “M propone”, cada mujer termina con su **peor** pareja válida (la más baja en su lista, entre todas las válidas para ella).

Demostración por absurdo. Supongamos que existe un par (m, w) en el matching final S^* donde m **no** es la peor pareja válida de w . Entonces existe algún matching estable S en el que w está emparejada con un hombre m' al que w prefiere *estrictamente menos* que a m (si m no es su peor válido, hay un válido peor que m , y ese es el m' que le toca en algún S estable).

Sea w' la pareja de m en S (existe porque S es perfecto y m aparece en él, ya que w es pareja válida de m). Por el Teorema de M-optimalidad, m está emparejado en S^* — el matching devuelto por Gale-Shapley — con **su mejor pareja válida**, y por hipótesis esa pareja es w . Como w' es también una pareja válida de m (está con él en el matching estable S) y w es la *mejor* válida de m , se tiene que **m prefiere w a w'** (o son la misma persona, caso trivial que no aporta al absurdo).

Ahora miro el par (m, w) dentro del matching estable S : m no está emparejado con w en S (está con w'), y por lo recién visto, m prefiere w a su pareja w' en S . Por otro lado, por hipótesis w prefiere m a su pareja m' en S . Entonces **(m, w) es una inestabilidad en S** : ambos se prefieren mutuamente sobre sus parejas en S . Esto contradice que S es un matching estable. ■

La demostración de W -pesimalidad se apoya **directamente** en el Teorema de M -optimalidad ya probado — no es independiente. La estructura es: “si w no tuviera a su peor pareja válida, se podría construir una inestabilidad en algún otro matching estable S , usando que la pareja de m en S^* (que es w) es la *mejor* válida para m por el teorema anterior”. Citar el teorema de M -optimalidad explícitamente como paso intermedio es lo que un corrector de tolerancia cero espera ver.

Ejemplo 5.1 — Verificación completa con simulación a mano, 4 hombres y 4 mujeres (misma instancia del Ejemplo 3.1), incluyendo M -optimalidad, W -pesimalidad y comparación M -propone vs. W -propone.

Preferencias:

m_1 : $w_1 > w_2 > w_3 > w_4$ m_2 : $w_1 > w_3 > w_2 > w_4$ m_3 : $w_2 > w_1 > w_4 > w_3$ m_4 : $w_2 > w_4 > w_1 > w_3$
 w_1 : $m_4 > m_3 > m_2 > m_1$ w_2 : $m_3 > m_4 > m_1 > m_2$ w_3 : $m_1 > m_2 > m_3 > m_4$ w_4 : $m_2 > m_1 > m_4 > m_3$

Paso 1 — Enumeración por fuerza bruta (script Python) de los 24 matchings perfectos posibles: se verificó, chequeando la definición de inestabilidad par por par en cada uno de los 24, que exactamente **2** son estables:

$S_1 = \{m_1-w_3, m_2-w_1, m_3-w_2, m_4-w_4\}$

$S_2 = \{m_1-w_3, m_2-w_4, m_3-w_2, m_4-w_1\}$

Paso 2 — Ejecutar “ M propone” (traza ya mostrada en el Ejemplo 3.1): el algoritmo devuelve exactamente $S_1 = \{m_1-w_3, m_2-w_1, m_3-w_2, m_4-w_4\}$.

Paso 3 — Ejecutar “ W propone” (mismas preferencias, roles invertidos) — traza:

1. w_1 propone a $m_4 \rightarrow$ libre $\rightarrow$ se comprometen (w_1, m_4)
2. w_2 propone a $m_3 \rightarrow$ libre $\rightarrow$ se comprometen (w_2, m_3)
3. w_3 propone a $m_1 \rightarrow$ libre $\rightarrow$ se comprometen (w_3, m_1)
4. w_4 propone a $m_2 \rightarrow$ libre $\rightarrow$ se comprometen (w_4, m_2)

Termina en la primera ronda (nadie es rechazado): devuelve $S_2 = \{m_1-w_3, m_2-w_4, m_3-w_2, m_4-w_1\}$.

Paso 4 — Verificación de M -optimalidad: para cada hombre, calculo su mejor pareja *válida* entre los dos matchings estables $\{S_1, S_2\}$ y comparo contra lo que le dio “ M propone” (S_1):

- m1: parejas válidas = {w3} (en ambos S_1 y S_2 tiene w3) $\rightarrow$ mejor = w3. GS le dio w3. ✓
- m2: parejas válidas = {w1 (en S_1), w4 (en S_2)}. m2 prefiere $w1 > w4 \rightarrow$ mejor = w1. GS (M propone) le dio w1. ✓
- m3: parejas válidas = {w2} en ambos $\rightarrow$ mejor = w2. GS le dio w2. ✓
- m4: parejas válidas = {w4 (en S_1), w1 (en S_2)}. m4 prefiere $w4 > w1 \rightarrow$ mejor = w4. GS le dio w4. ✓

Los 4 hombres reciben, con “M propone”, exactamente su mejor pareja válida.

Paso 5 — Verificación de W-pesimalidad (comparando contra el mismo matching S_1 = resultado de “M propone”):

- w1: parejas válidas = {m2 (en S_1), m4 (en S_2)}. w1 prefiere $m4 > m3 > m2 > m1 \rightarrow$ la peor de sus válidas es m2. “M propone” le dio m2. ✓
- w2: parejas válidas = {m3} en ambos $\rightarrow$ peor = m3. Le dio m3. ✓
- w3: parejas válidas = {m1} en ambos $\rightarrow$ peor = m1. Le dio m1. ✓
- w4: parejas válidas = {m4 (en S_1), m2 (en S_2)}. w4 prefiere $m2 > m1 > m4 > m3 \rightarrow$ la peor de sus válidas es m4. Le dio m4. ✓

Las 4 mujeres reciben, con “M propone”, exactamente su peor pareja válida.

Conclusión verificada: “M propone” da a los hombres el mejor resultado posible y a las mujeres el peor, dentro de lo que la estabilidad permite — exactamente lo que enuncian los dos teoremas, confirmado por fuerza bruta sobre las 24 asignaciones posibles y no solo por argumento teórico.

Truco de examen: si el enunciado dice que “M propone” y “W propone” dan el **mismo** matching, eso es evidencia (no prueba automática, pero fuerte indicio) de que la instancia tiene un **único** emparejamiento estable — en ese caso, la mejor y la peor pareja válida de cada persona coinciden (solo hay una), así que M-optimalidad y W-pesimalidad “colapsan” al mismo resultado. Es un buen chequeo rápido antes de gastar tiempo enumerando todos los matchings estables a mano.

6. Complejidad temporal: por qué es exactamente $O(n^2)$

Por qué NO es $O(n)$: el tamaño de la entrada ya es $\Theta(n^2)$ — cada una de las n personas trae una lista de preferencia de tamaño n , así que hay $2n$ listas de n elementos cada una. Ningún algoritmo puede correr en menos tiempo que el necesario para *leer* su entrada, y aquí la entrada mide $\Theta(n^2)$. Además, el Ejemplo 3.1 (y la instancia clásica de Kevin Wayne que alcanza exactamente $n(n-1)+1$ propuestas) muestra que existen instancias donde el algoritmo efectivamente necesita un número de propuestas del orden de n^2 , no de n .

Por qué NO es $O(n^3)$ (con una implementación correcta): la cota de n^2 *propuestas* del ciclo `while` (Sección 3) da el número de iteraciones. La pregunta es cuánto cuesta **cada** iteración:

- Elegir la siguiente mujer a la que m le propone: $O(1)$ si se mantiene, para cada hombre, un puntero al próximo lugar de su lista de preferencia (Sección de “Aplicación eficiente” del teórico).
- Decidir si w prefiere a m sobre su pareja actual m' : esto **parece** requerir recorrer la lista de preferencia de w (costo $O(n)$ por consulta) — si se implementa así ingenuamente, el costo total sería n^2 iteraciones $\times O(n)$ por iteración = $O(n^3)$, y ese es precisamente el error de implementación de tolerancia cero que hay que evitar.
- La solución (usada en el teórico del curso) es precomputar, para cada mujer, un **array inverso de preferencia**: `inversa[m] = posición de m en la lista de w`. Ese precómputo cuesta $O(n)$ por mujer, $O(n^2)$ en total para las n mujeres, y una vez armado, comparar “¿ w prefiere m a m' ?” es $O(1)$: simplemente `inversa[m] < inversa[m']`.

Con esa estructura de datos, cada iteración del `while` cuesta $O(1)$, hay a lo sumo n^2 iteraciones, y el precómputo inicial cuesta $O(n^2)$ — todo suma $O(n^2)$ en total, ni más ni menos.

Error frecuente de tolerancia cero: justificar la complejidad del algoritmo citando *solo* la cota de n^2 iteraciones del `while`, sin analizar el costo de “¿ w prefiere m a m' ?” dentro de cada iteración. Si esa consulta no está justificada como $O(1)$ (vía el array inverso), la complejidad total declarada como $O(n^2)$ es un salto en el razonamiento — un corrector de tolerancia cero pide explícitamente esa justificación, porque la implementación ingenua da $O(n^3)$.

7. Variantes típicas de examen

El patrón detectado sobre exámenes y parciales del curso (relevado en la guía semanal) es que Emparejamiento Estable **casi nunca** se evalúa como “correr el algoritmo tal cual” — lo típico es una variante que exige adaptar el algoritmo y **rehacer las cuatro demostraciones** (termina, es matching, es perfecto — o casi-perfecto —, es estable) con la definición de inestabilidad modificada.

7.A) PARES PROHIBIDOS / LISTAS INCOMPLETAS (ACEPTABILIDAD)

Generalización. Existe un subconjunto $F \subseteq M \times W$ de pares que **no pueden casarse** (por ejemplo, un aplicante no calificado para cierta empresa). Cada persona solo lista, en su preferencia, a quienes le son aceptables (es decir, con quienes NO forma un par prohibido). Bajo esta generalización, un matching estable **no necesita ser perfecto** — puede haber gente sin pareja, siempre que sea porque ya agotó su lista de aceptables.

Definición de inestabilidad con 4 casos (Ejercicio 2, Práctico 1): dado un matching S , hay una inestabilidad si ocurre alguno de:

1. (m, w) y $(m', w') \in S$ con $(m, w') \notin F$, m prefiere w' a w , y w' prefiere m a m' (el tipo usual).
2. $(m, w) \in S$ y existe m' soltero, con $(m', w) \notin F$, tal que w prefiere m' a m (una persona soltera y no-prohibida es “más deseable” que quedarse con quien no la prefiere).

3. Simétrico al anterior, con una mujer soltera w' que m prefiere y $(m, w') \notin F$.
4. Un hombre m y una mujer w , ambos solteros, con $(m, w) \notin F$ (dos solteros sin nada que se lo impida deberían juntarse).

Adaptación del algoritmo: se modifica solo la condición del `while`, agregando el filtro de aceptabilidad: *"Mientras exista un hombre m libre que no le haya propuesto a toda mujer w para la cual $(m, w) \notin F$ ".* El resto del algoritmo queda igual.

Resultado (Práctico 1, ejercicio resuelto): esta variante siempre tiene solución estable — se demuestra adaptando exactamente la misma estructura de las 4 demostraciones de la Sección 4, reemplazando "propuso a todas las mujeres" por "propuso a todas las mujeres no-prohibidas", y probando por absurdo que ninguno de los 4 tipos de inestabilidad puede ocurrir en el matching devuelto (usando, en cada caso, que quien pudo haber causado la inestabilidad ya había sido rechazado — o nunca propuso — durante la ejecución).

7.B) CUPOS MÚLTIPLES (HOSPITALES-RESIDENTES, KT EX. 1.4)

Cada hospital h tiene una capacidad q_h (puede admitir hasta q_h residentes, no solo 1). La definición de inestabilidad cambia: (h, s) es inestable si h y s son aceptables entre sí, **y** (s no tiene hospital asignado, o prefiere h al que tiene), **y** (h tiene una vacante libre, o prefiere a s sobre alguno de los residentes que ya tiene asignados). El algoritmo se adapta para que cada hospital pueda tener hasta q_h "compromisos" simultáneos, comparando cualquier propuesta nueva contra el **peor** de sus comprometidos actuales cuando ya está lleno. Las cuatro demostraciones se rehacen con la misma estructura, pero " m' vuelve a estar libre" pasa a ser "el peor residente asignado a h se libera".

7.C) ORDEN DE PROPUESTA O FUNCIÓN DE COMPARACIÓN NO ESTÁNDAR

Variantes vistas en exámenes (2021/12, 2023/02): en vez de una lista de preferencia explícita, se da una función de costo o "ranking" `pref(x, y)` que hay que usar para comparar, y a veces el orden en que se recorre a los libres no es estrictamente "cualquier hombre libre" sino alguna regla adicional (por rondas, con prioridades). La receta para estos ejercicios es: (1) identificar cuál es el rol de " M " (quien propone) y cuál el de " W " (quien acepta/rechaza); (2) verificar que las dos invariantes de la Sección 2 siguen valiendo bajo la nueva regla; (3) si valen, las cuatro demostraciones de las Secciones 3 y 4 se trasladan casi literalmente, cambiando solo la cota de terminación si el espacio de "propuestas posibles" cambió de tamaño.

7.D) REDUCCIONES A EMPAREJAMIENTO ESTABLE

Aparece también como **herramienta de modelado**: un problema de otro dominio se resuelve *reduciéndolo* a una instancia de emparejamiento estable y aplicando Gale-Shapley (o probando que no siempre hay solución, con un contraejemplo).

Ejemplo 7.1 — Reducción (KT Ex. 1.6, Peripatetic Shipping Lines).

n barcos, n puertos, cada barco visita cada puerto exactamente un día del mes; nunca dos barcos en el mismo puerto el mismo día. Se busca asignar a cada barco un puerto de "mantenimiento" (donde queda varado el resto del mes) tal que nunca haya dos barcos en el mismo puerto el mismo día.

Modelado: B = barcos, P = puertos, buscar un emparejamiento perfecto (b_i, p_i) . La clave (dada en la sugerencia del ejercicio) es definir las preferencias así: cada barco “prefiere” los puertos en el **orden en que los visita** en su agenda (antes = preferido), y cada puerto “prefiere” a los barcos en el **orden inverso de llegada** (el que llega último es el preferido). Con esta construcción, una inestabilidad correspondería exactamente a la situación prohibida: un barco b_1 varado en p_1 mientras otro barco b_2 iba a visitar p_1 *antes* de llegar a su propio puerto de mantenimiento — Gale-Shapley garantiza que esa situación no ocurre en el matching estable resultante.

Conclusión: como el problema de emparejamiento estable siempre tiene solución (Sección 4), y esta construcción particular hace que “estable” equivalga exactamente a “cumple el requisito de seguridad de PSL”, la existencia de una planificación válida queda demostrada, y el algoritmo para encontrarla es directamente Gale-Shapley sobre esta instancia construida.

No se encontraron errores conceptuales en el material fuente del curso (teórico resumen, diapositivas, prácticos resueltos) al contrastarlo con el libro y con la simulación numérica. Los apuntes manuscritos del teórico usan una notación levemente distinta a las diapositivas en inglés/español del curso (p. ej. `best(m)` vs. “mejor pareja válida”), pero son consistentes entre sí en el contenido matemático.

Tabla de referencia rápida — objetos del problema

Objeto	Definición en una línea
Matching	Subconjunto de $M \times W$ sin personas repetidas
Matching perfecto	Matching de tamaño n (nadie sin pareja)
Inestabilidad / rogue couple	Par (m, w) fuera de S donde AMBOS prefieren al otro sobre su pareja actual
Matching estable	Perfecto + sin inestabilidades
Pareja válida	w es válida para m si (m, w) están juntos en ALGÚN matching estable
Asignación M-óptima	Cada m con su mejor pareja válida
Asignación W-pesimal	Cada w con su peor pareja válida
Cota de propuestas	A lo sumo n^2 (demostrado contando pares (m, w) posibles)
Complejidad total	$O(n^2)$, con array inverso de preferencia para comparar en $O(1)$

Ejercicios de repaso rápido (con respuesta)

#	Pregunta	Respuesta
1	¿Qué dos condiciones tiene que cumplir un matching para ser estable?	Ser perfecto Y no tener inestabilidades
2	¿La definición de inestabilidad usa AND u OR entre las dos preferencias?	AND — ambos deben preferirse mutuamente
3	¿Cuál es la cota de propuestas de Gale-Shapley y de dónde sale?	A lo sumo n^2 ; sale de contar los pares (m,w) posibles, cada uno usado como propuesta a lo sumo una vez
4	¿Qué le pasa a una mujer una vez que queda comprometida?	Nunca vuelve a estar libre; solo mejora de pareja
5	¿Qué le pasa a un hombre libre con el tiempo?	Solo empeora: cada rechazo lo manda a proponerle a alguien peor en su propia lista
6	En “M propone”, ¿quién recibe la mejor asignación posible?	Los hombres (M-optimalidad): cada uno con su mejor pareja válida
7	En “M propone”, ¿quién recibe la peor asignación posible (dentro de lo estable)?	Las mujeres (W-pesimalidad): cada una con su peor pareja válida
8	¿Por qué el algoritmo NO es $O(n)$?	La entrada ya mide $\Theta(n^2)$ ($2n$ listas de tamaño n)
9	¿Por qué el algoritmo NO es $O(n^3)$ en una implementación correcta?	Con array inverso de preferencia, “¿w prefiere m a m’?” es $O(1)$, no $O(n)$
10	¿Qué error de implementación produce $O(n^3)$ en vez de $O(n^2)$?	Comparar preferencias recorriendo la lista completa en cada consulta, en vez de precomputar el array inverso
11	En la variante de pares prohibidos, ¿el matching estable resultante es siempre perfecto?	No necesariamente — puede haber gente sin pareja si agotó su lista de aceptables
12	¿Cuántos tipos de inestabilidad hay que chequear en la variante con pares prohibidos?	4 (el tipo usual + 2 variantes con alguien soltero + 2 solteros mutuamente aceptables)

Resumen ultra-rápido (2 minutos)

- **Instancia:** M y W de tamaño n , listas de preferencia totales y sin empates.
- **Estable = perfecto + sin inestabilidades.** Inestabilidad = par (m,w) fuera de S donde AMBOS se prefieren mutuamente sobre su pareja actual (AND, no OR).
- **Gale-Shapley “M propone”:** mientras haya un hombre libre con mujeres pendientes, le propone a la mejor no intentada; ella acepta si está libre o prefiere a m sobre su pareja actual, si no lo rechaza.
- **Invariantes:** hombres proponen en orden decreciente (solo empeoran su aspiración); mujeres solo mejoran una vez comprometidas, nunca vuelven a estar libres.

- **Termina** en a lo sumo n^2 iteraciones (cota por conteo de pares (m,w) posibles).
- **Es matching, es perfecto:** por absurdo, usando que un hombre libre siempre tiene mujeres pendientes mientras el algoritmo no terminó.
- **Es estable:** por absurdo, asumiendo una inestabilidad (m,w) y usando que si w no terminó con m fue porque lo rechazó o lo dejó por alguien que le gusta más — contradicción con que w prefiere m a su pareja final.
- **M-optimalidad / W-pesimalidad:** “M propone” da a cada hombre su mejor pareja válida y a cada mujer su peor pareja válida, entre todos los matchings estables posibles de la instancia — demostrado y verificado numéricamente por fuerza bruta en el Ejemplo 5.1.
- **Complejidad $O(n^2)$:** n^2 iteraciones $\times O(1)$ por iteración (con array inverso de preferencia precomputado en $O(n^2)$). Sin ese array, la implementación ingenua da $O(n^3)$ — error de tolerancia cero.
- **Variantes de examen** (lo más frecuente, no el caso base): pares prohibidos/listas incompletas (matching no necesariamente perfecto, 4 tipos de inestabilidad), cupos múltiples (Hospitales-Residentes), función de preferencia no estándar, y reducciones de otros problemas a Emparejamiento Estable.